semi-PD: TOWARDS EFFICIENT LLM SERVING VIA PHASE-WISE DISAGGREGATED COMPUTATION AND UNIFIED STORAGE

Ke Hong[1,2] **Lufang Chen**[2] **Zhong Wang**[2] **Xiuhong Li**[2] ✉ **Qiuli Mao**[2] **Jianping Ma**[2]

Chao Xiong[2] **Guanyu Wu**[1,2] **Buhe Han**[2] **Guohao Dai**[2,3] ✉ **Yu Wang**[1] ✉

✉ lixiuhong@infini-ai.com, daiguohao@sjtu.edu.cn, yu-wang@tsinghua.edu.cn

ABSTRACT

Existing large language model (LLM) serving systems fall into two categories: 1) a unified system where *prefill* phase and *decode* phase are co-located on the same GPU, sharing the unified computational resource and storage, and 2) a disaggregated system where the two phases are disaggregated to different GPUs. The design of the disaggregated system addresses the latency interference and sophisticated scheduling issues in the unified system but leads to storage challenges including 1) replicated weights for both phases that prevent flexible deployment, 2) KV cache transfer overhead between the two phases, 3) storage imbalance that causes substantial wasted space of the GPU capacity, and 4) suboptimal resource adjustment arising from the difficulties in migrating KV cache. Such storage inefficiency delivers poor serving performance under high request rates.

In this paper, we identify that the advantage of the disaggregated system lies in the disaggregated computation, *i.e.*, partitioning the computational resource to enable the asynchronous computation of two phases. Figure 1 illustrates the pros and cons of the different computation and storage patterns. Thus, we propose a novel LLM serving system, *semi-PD*, characterized by disaggregated computation and unified storage. In *semi-PD*, we introduce a computation resource controller to achieve disaggregated computation at the streaming multi-processor (SM) level, and a unified memory manager to manage the asynchronous memory access from both phases. *semi-PD* has a low-overhead resource adjustment mechanism between the two phases. On top of such an adjustment mechanism, we further design a service-level objective (SLO) aware dynamic partitioning algorithm to optimize the SLO attainment. The evaluation shows that compared to state-of-the-art systems, *semi-PD* maintains lower latency at higher request rates, reducing the average end-to-end latency per request by 1.27-2.58× on DeepSeek series models, and serves 1.55-1.72× more requests adhering to latency constraints on Llama series models.

1 Introduction

With the rapid development of large language models, more and more users are beginning to adopt large language model services for content generation, in the domains including chatbots [1, 2, 3], code assistants [4, 5], and agent-based applications [6, 7, 8]. In LLM serving, users typically send requests to large model service providers and wait for a response. However, LLM service tends to introduce a considerable latency due to the tremendous computation and memory access requirements [9]. Unlike traditional requests, the latency of LLM requests consists of two distinct parts. This is because each LLM request involves two processing phases: the *prefill* phase and the *decode* phase. The *prefill* phase processes the input prompt and generates the first output token, and the *decode* phase generates the output

[1] Tsinghua University
[2] Infinigence-AI
[3] Shanghai Jiao Tong University
✉ Corresponding authors: Yu Wang, Xiuhong Li, Guohao Dai.

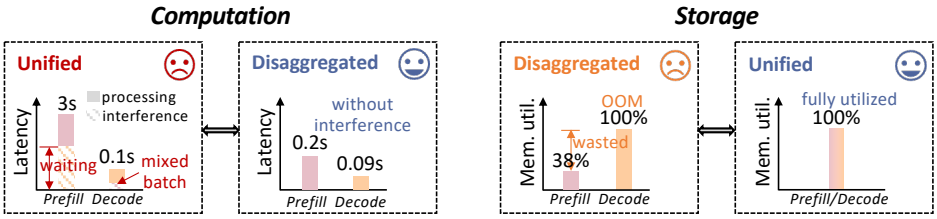

Figure 1: Illustration of the pros and cons of the different computation and storage patterns. *semi-PD* can have the advantages of both disaggregated computation and unified storage.

tokens iteration by iteration in an auto-regressive manner, with one token at each iteration. Thus, the first part is the Time-to-First-Token (TTFT), which measures the time between the request arrival and the first output token being generated via the *prefill* phase. The second part is the Time-per-Output-Token (TPOT), which measures the average latency per token in the *decode* phase. Constraining TTFT and TPOT, as the two components of the service-level objective (SLO), are both essential for user experience. Thus, adhering to the TTFT and TPOT constraints, how to schedule the *prefill* and *decode* phase of each request to maximize the system throughput is very important.

LLM request batching is an effective method for improving system throughput. First of all, to solve the challenge that the LLM requests have different input prompt lengths and different output generation lengths, Orca [10] proposes the continuous batching technique, scheduling execution and batching requests at the granularity of iterations (an execution of *prefill* phase or *decode* phase) instead of requests. Based on the iterative batching mechanism, many works focus on how to schedule *prefill* iterations and *decode* iterations. Researchers first choose to place *prefill* and *decode* iterations within the same GPU as shown in Figure 2(a). They either prioritize the *prefill* iterations (vLLM [11]) or prioritize the *decode* iterations (FasterTransformer [12]). The former benefits TTFT but at the cost of TPOT, and vice versa. Such a trade-off is called the latency interference between the two phases. Therefore, to alleviate the interference of TTFT on TPOT, Sarathi-Serve [13] and DeepSpeed-FastGen [14] propose to chunk the input of *prefill* request into smaller *prefill* segments and then schedule the *prefill* segments together with *decode* requests in one batch. However, the aforementioned works cannot avoid the interference between TTFT and TPOT, and it is very challenging to meet both TTFT and TPOT constraints in SLO with an optimal system throughput. To this end, researchers propose the disaggregated design which places the *prefill* iteration and *decode* iteration on different GPUs to eliminate the interference between TTFT and TPOT [15, 16, 17, 18, 19]. In the disaggregated system, some GPUs act as the *prefill* instance, and others act as the *decode* instance, as shown in Figure 2(b).

In this paper, we use the term storage to denote the memory capacity on GPUs. We identify the drawbacks of the disaggregated design caused by its disaggregated storage for the model weights and KV cache. 1) **Storage imbalance.** The *prefill* instance only generates the part of the KV cache corresponding to the input length. Meanwhile, the *decode* instance needs to accommodate the whole KV cache corresponding to the full sequence length including the output length. In cases where the output length is larger, the storage imbalance will become more severe. Thus, the storage capacity required by the *decode* instance is significantly larger than that of the *prefill* instance. 2) **KV cache transfer.** Since the *prefill* phase and *decode* phase are placed on different GPUs, it is necessary to transfer the KV cache from the *prefill* instance to the *decode* instance. The transfer overhead becomes significant on low-end GPUs due to communication latency, necessitating careful parallelism strategy designs [17]. Even for the high-end GPUs equipped with NVLink [20], the transfer time will be calculated into TPOT, making TPOT sub-optimal. 3) **Resource adjustment between *prefill* and *decode* instances.** The workload of *prefill* and *decode* requests varies as the serving process goes on. However, the GPU-level disaggregation results in coarse-grained adjustment. Moreover, since the KV cache is mainly located on the *decode* instance, the adjustment needs specific designs to avoid the cost of KV cache transfer, which introduces additional overhead. 4) **Weight replica.** The *prefill* instance and *decode* instance both need to accommodate the whole model weight. Disaggregated systems face deployment challenges when the number of available GPUs is limited.

We comprehensively compare and analyze the unified design and disaggregated design, as shown in Figure 2(a) and Figure 2(b). The key insight is that we should adopt the disaggregated computation feature of the disaggregated system and reserve the unified storage feature of the unified system. Moreover, the system is supposed to be lightweight and support adjusting the resource ratio between *prefill* and *decode* phases with negligible overhead, thereby adaptively matching the workload changes during serving. In this paper, we propose *semi-PD*, an efficient LLM serving system characterized by phase-wise disaggregated computation and unified storage. The design of *semi-PD* consists of 1) a computation resource controller to support disaggregated computation and lightweight resource adjustment, with the disaggregated computation of each phase abstracted as a worker, and 2) a unified memory manager to coordinate the

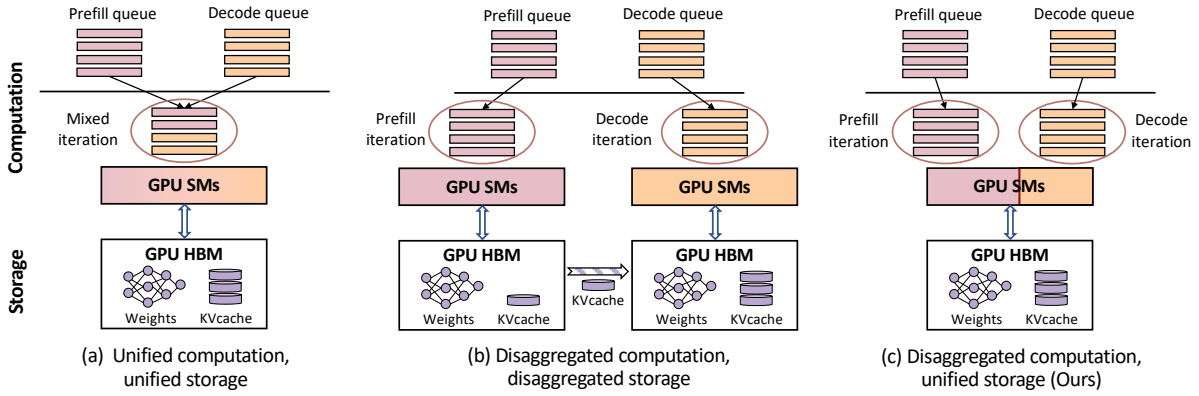

Figure 2: Comparison among different computation and storage design paradigms for LLM serving.

memory accesses of model weights and KV cache for both *prefill* and *decode* workers. Based on that, we orchestrate a low-overhead dynamic resource-adjusting algorithm based on the SLO requirements. Finally, we implement *semi-PD* and perform a comprehensive evaluation to compare with the state-of-the-art works.

In summary, this paper makes the following contributions:

- We comprehensively analyze the unified and disaggregated designs and propose a phase-wise disaggregated computation and unified storage design to improve serving performance. We further build a system prototype that fully matches the proposed design.
- Our system supports adjusting the resource ratio between *prefill* and *decode* workers with a low-overhead worker switching mechanism and SLO-aware switching algorithm.
- We implement *semi-PD*, and evaluate across various benchmarks. The results demonstrate that *semi-PD* reduces the request-level latency by 1.27-2.58×, and serves 1.55-1.72× more requests, compared to state-of-the-art systems while staying within latency constraints for > 90% of requests.

2 Background

2.1 LLM Processing Phases

Modern LLMs [8, 21] utilize transformer-based [22] decoder-only [23] architecture to predict the next token in an autoregressive manner. The LLM processing can be divided into two phases: *prefill* phase and *decode* phase. Such design benefits from the KV cache that stores the information of previous tokens to avoid repeated computation. During the *prefill* phase, the model processes the input context, generates the initial KV cache for each input token, and derives the first output token. During the *decode* phase, the model produces the next token based on the last generated token and the KV cache. Meanwhile, the newly generated key and value vectors of the current input token are appended to the KV cache. The LLM iteratively generates output tokens until reaches a pre-defined maximum token length or an end-of-sequence (<*eos*>) token.

2.2 LLM Serving

In LLM serving, users interact with the LLM by sending requests (*e.g.*, text inputs) and receiving responses (*e.g.*, generated text).

2.2.1 Latency and SLOs.

Commonly used latency metrics for LLM serving include TTFT and TPOT.

- Time-to-First-Token (TTFT) is the duration between the arrival time and the time when the *prefill* phase finishes of a request.
- Time-per-Output-Token (TPOT) is the average duration to generate the subsequent tokens (except for the first one) of a request. Some works [19] use Time-between-Tokens (TBT) to measure the latency between successive tokens. In this work, we use the TPOT metric.

TTFT and TPOT capture the request-level latency experienced by users. To ensure the user experience, the agreement between service providers and users determines SLOs (*e.g.*, TTFT $\leq$ 500ms, TPOT $\leq$ 100ms) as the latency constraints. To address the importance of SLOs, many works [24, 17, 25, 19] consider serving more requests while adhering to the SLOs, which is also the objective of this work.

2.2.2 Batching.

Continuous batching is the basic technique in LLM serving. The differences in the output length prevent the requests in a batch from being finished at the same time. The ended requests are not returned to users immediately, and the incoming requests are forced to wait until the current batch is fully processed. That kind of request-level batching harms both system-level utilization and request-level latency. To address the issue, the technique of continuous batching is introduced in Orca [10], which batches the requests at the iteration level. With continuous batching, at each iteration, the finished requests exit, and the incoming requests are added to the current batch. The continuous batching technique is widely used in various serving systems, including both unified and disaggregated designs.

2.3 Related Works

Based on the continuous batching technique, different LLM serving designs have been proposed. In this section, we discuss the related works including the phase-wise unified and disaggregated system designs. Moreover, we also list the related works involving request prioritization and throughput-oriented optimization, which are orthogonal to this work.

2.3.1 Unified system.

In a unified system, the works with batching and scheduling designs mainly focus on the trade-offs between *prefill* phase and *decode* phase. The default scheduling strategy in vLLM [11] prioritizes the *prefill* phase of new requests over the *decode* phase of the running ones to improve the throughput via larger batch size, but at the cost of TPOT degrading drastically. FasterTransformer [12] prioritizes *decode* iteration, leading to better TPOT at the cost of worse TTFT and system throughput. To mitigate such degradation, Sarathi-Serve [24] and DeepSpeed-FastGen [14] propose the SplitFuse method to chunk the *prefill* requests along the sequence dimension and schedule the *decode* requests together with the chunked *prefill* requests within one batch. Besides, SGLang [26], TensorRT-LLM [27], and LMDeploy [28] are all highly optimized unified systems for LLM serving. Nevertheless, all of those implementations fail to eliminate the latency interference.

2.3.2 Disaggregated system.

The disaggregated design removes the *prefill-decode* interference by disaggregating the *prefill* and *decode* phases to different GPUs. Based on the disaggregated design, Splitwise [16] further utilizes different types of GPU for different phase deployment and maintains a mixed machine pool to realize the resource adjustment between phases. DistServe [17] develops algorithms to derive the optimal parallelism separately for *prefill* phase and *decode* phase and reruns the placement algorithm periodically to suit the workload changes. TetriInfer [15] presents a detailed top-down system to discuss the design against various workloads and use an instance flip mechanism for resource adjustment between the two phases. ExeGPT [18] proposes different scheduling strategies involving assigning the execution of *prefill* and *decode* phases to different GPUs, and discusses the performance under the latency constraint. Mooncake [19] and MemServe [29] both unveil the potential of disaggregated systems combined with the prefix caching technique [26]. LoongServe [25] introduces elastic sequence parallelism, where each machine stores a portion of the KV cache, supporting more flexible dynamic scaling up or down. We observe the storage challenges related to all those disaggregated systems, and pose an analysis in Section 3.

2.3.3 Request prioritization.

The intra-phase request prioritization also significantly impacts performance, particularly in terms of latency. FastServe [30] uses preemptive scheduling to minimize the total latency, giving shorter requests a higher priority to minimize the total latency. [31] defines the fairness between users and treats the requests from different users with different priorities. [32] proposes prioritizing requests with shorter estimated computation time, aiming to reduce the average queuing time.

2.3.4 Throughput-oriented optimization.

Those works mainly focus on enlarging the batch size or improving the hardware utilization, and are not engaged in latency objectives or SLOs. The paged KV cache management in vLLM [11] significantly improves the throughput

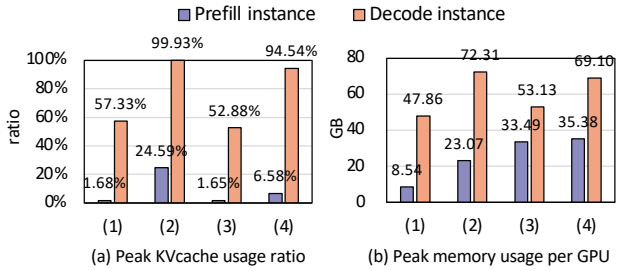

Figure 3: Storage imbalance between *prefill* instance and *decode* instance. (1) and (2) are measured with Llama3-8B, on ShareGPT and LongBench datasets, respectively. We set TP=2/1 for *prefill*/*decode* instances. (3) and (4) are measured with Llama3-70B, on ShareGPT and LongBench datasets, respectively. We set TP=4 for both instances.

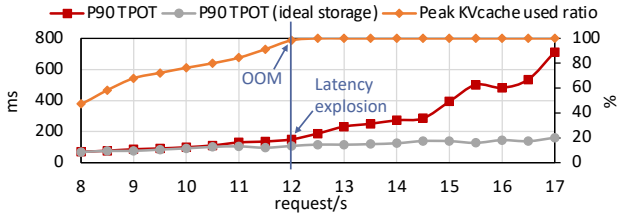

Figure 4: The consistency between storage shortage and latency explosion. The data is measured on an A100 40GB GPU, and the reported request rate is per GPU. P90 TPOT increases drastically when the storage space is exhausted. For comparison, we draw the P90 TPOT curve under the ideal storage.

of the LLM serving system, by removing the fragments caused by incontiguous storage. NanoFlow[33] proposes fine-grained intra-device parallelism, offering operational-level scheduling to maximize hardware efficiency. Other works [27, 34, 35, 36] investigate the faster implementation of GPU kernels, which also contribute to the throughput enhancement.

3 Analysis and Motivation

The advantage of disaggregated computation lies in its ability to eliminate the latency interference between *prefill* and *decode* phases, thereby enabling a simplified and phase-specific scheduling strategy, as extensively investigated in prior disaggregated system research [16, 17, 19]. In this section, we quantify the drawbacks of disaggregated storage from the four aforementioned aspects: 1) storage imbalance, 2) KV cache transfer overhead, 3) non-negligible worker adjustment overhead, and 4) weight replica, thereby demonstrating that unified storage can be a beneficial solution.

3.1 Storage Imbalance

The whole KV cache is stored in the *decode* instance for disaggregated systems. Meanwhile, the *prefill* instance only holds the initial KV cache generated by the *prefill* phase, and transfers it immediately after *prefill* phase finishes. Such a mechanism leads to the KV cache storage imbalance between the two instances. We illustrate the imbalance on typical benchmarks in Figure 3. At most, only 25% of the KV cache is occupied for the *prefill* instance, while all of the KV cache can be utilized for the *decode* instance. Although DistServe [17] proposes to use a pull-based KV cache transfer for the *decode* instance to alleviate the imbalance, only a small proportion of KV cache stays on the *prefill* instance. As a result, up to 89.33% of the GPU memory space will be wasted on the *prefill* instance and the disaggregated storage disables the utilization of such wasted memory space.

Such inefficient storage causes the *decode* instance to run out of memory (OOM) early, leading to latency explosion due to waiting or preemption. We present such causality in Figure 4 by showing the consistency between storage shortage and TPOT explosion. The storage shortage leads to 3× higher TPOT, significantly harming the serving efficiency.

3.2 KV Cache Transfer Overhead

Since the *prefill decode* phases are disaggregated to different GPUs, the *prefill* instance needs to transfer the generated KV cache to the *decode* instance for computation. DistServe [17] and Splitwise [16] report that the KV cache transfer

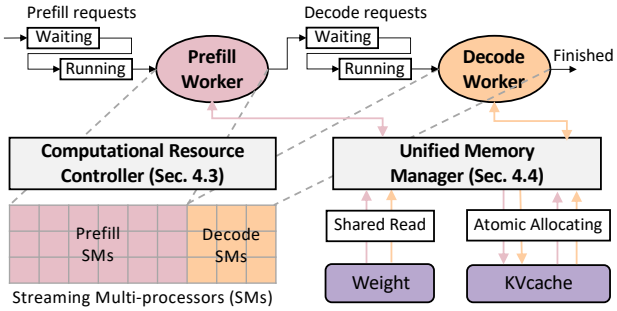

Figure 5: System overview of *semi-PD*.

overhead is negligible compared to the end-to-end latency, based on the claim that the transfer happens only once, but the token generation repeats until the <*eos*> token or the maximum token length is met. Splitwise [16] introduces the layer-wise transfer to overlap the KV cache transfer overhead and the *prefill* computation. Although KV cache transfer latency does not constitute the primary bottleneck in disaggregated systems, it nevertheless introduces non-trivial complexity to system design, including communication pattern and parallelism strategy. Besides, we observe that the KV cache transfer overhead is comparable to the latency of a single token generation, indicating that the second output token suffers a multiple times longer delay than the others.

3.3 Resource Adjustment Overhead

Most disaggregated systems are coupled with resource adjustment mechanisms. While unified systems naturally support resource adjustment between the *prefill* and *decode* phases, disaggregated systems require additional design to facilitate such adjustments due to the disaggregation of resources. There are three ways to perform resource adjustment between phases: 1) periodic replanning in DistServe [17] that generates the new parallelism pattern to adjust the resource partition between *prefill* and *decode* phases, 2) mixed machine pool in Splitwise [16] is maintained to perform mixed batching as needed, and 3) instance flip in TetriInfer [15] that flips an instance running low workloads.

We identify that the adjustment methods in disaggregated systems suffer from coarse-grained control and high overhead. All of 1), 2) and 3) make adjustments at the granularity of one GPU, together with its computational capacity and bandwidth. DistServe reports an overhead of several minutes with 1), including reloading weights, which is overwhelming in the real-time serving scenarios. With 2) the mixed batching of requests from different phases tends to cause latency interference again. The overhead of 3) comes from waiting for the existing queue to finish, which brings under-utilization as the batch size shrinks. The system chooses to drain the existing requests as it suffers more from frequent KV cache migration.

3.4 Replicated Weights

For disaggregated systems, both *prefill* and *decode* instances hold a copy of complete model weights on corresponding GPUs. The replicated weights disable the flexible deployment for disaggregated systems. For example, deploying Llama3-8B instances needs at least 2 GPUs. Two servers equipped with 16 A100 80GB GPUs can not hold a pair of *prefill* and *decode* instances for Llama3.1-405B. But a unified system still has 470GB of space left with such two servers after deployment. Thus, the disaggregated systems demand far more GPUs due to the replicated weights. However, the replicated instances necessitate the request rate to be large enough to fully utilize the GPU capacity, which is not always true in practice.

4 System Design

Based on the analysis in Section 3, the combination of disaggregated computation and unified storage takes the dominant advantages of the disaggregated and the unified systems. Therefore, we build an LLM serving system, *semi-PD*, characterized by disaggregated computation and unified storage. In this section, we describe the system design of *semi-PD*.

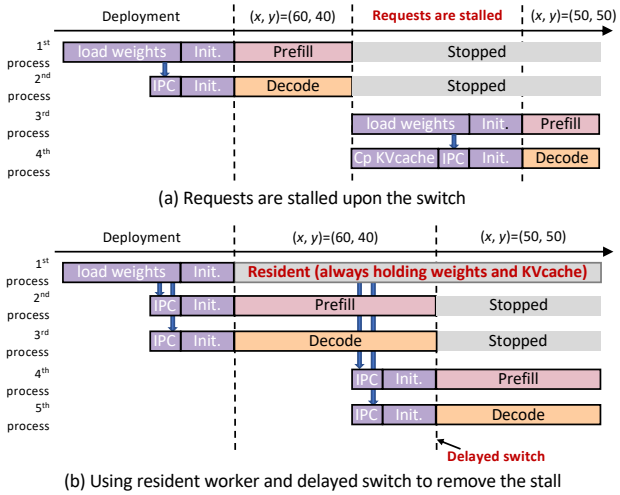

Figure 6: Low-overhead switch mechanism in *semi-PD*.

4.1 Overview

Figure 5 presents the system overview. The disaggregated computation of each phase is denoted as a worker, and the *prefill* and *decode* workers handle *prefill* and *decode* requests, respectively. Each worker maintains its own waiting and running queue for arrived requests. On top of the two workers, the disaggregated computation is achieved via a computational resource controller, which decides the computational resource partition between the two workers, and performs the real-time switching as needed. For the storage part, *semi-PD* manages the system's memory utilization via a unified memory manager, which controls both workers' memory access of weight and KV cache.

4.2 *Prefill* and *Decode* Workers

Same as the disaggregated designs, the *prefill* and *decode* workers are separated in *semi-PD*, each owning a single process and performing asynchronous computation independently. For a single request, it is first stacked into the waiting queue of the *prefill* worker upon arrival. The *prefill* worker schedules the request for running and performs the *prefill* iteration. Meanwhile, the generated K, V projection of the request is written into the KV cache. When *prefill* phase finishes, the request is stacked into the waiting queue of *decode* worker. The *decode* worker schedules it for running and performs the *decode* iteration repeatedly until it is finished. At each *decode* iteration, the KV cache of the request is updated.

4.3 Computational Resource Controller

Disaggregated computation. We implement the disaggregated computation based on the Multi-Process Service (MPS) CUDA application interface [37]. The MPS allows dispatching a certain number of SMs to a process, thereby achieving the computational resource partition from the SM level. *semi-PD* receives a (x, y) configuration, where x and y denote the percentage of the total SMs dispatched to the *prefill* and *decode* processes via MPS, respectively.

Resource adjustment: challenge. Then, we propose a lightweight resource adjustment mechanism. As mentioned in Section 4, each worker is tied to a process, and a pair of processes can be assigned with only one (x, y). MPS does not support adjusting (x, y) of the existing processes. Thus, adjusting (x, y) needs to reinvoke the MPS interface, thereby bringing overhead from the processes' switch. We need to wait for the *prefill* worker and *decode* worker to finish their running iteration and store the generated token. Then, we kill the two processes. Finally, we renew two processes using the MPS interface as the new *prefill* worker and *decode* worker with new (x, y). Based on the KV cache and the next token, the new workers go on. Thus, the overhead comes from two aspects. First, as shown in Fig. 6, renewing two MPS processes includes 1) loading the weights to the new *prefill* worker and sharing with the new *decode* worker via IPC (or vice versa), 2) copying the KV cache to the new *decode* worker, and 3) initializing the engine for both new workers. Such switching overhead causes the arrived requests to be stalled until the new workers are ready. Second, the switching timing is also important. Because *prefill* workers and *decode* workers have different processes, we need to insert a synchronous operation to stall the early finished worker, but the stalling behavior leads to an idle period for request processing.

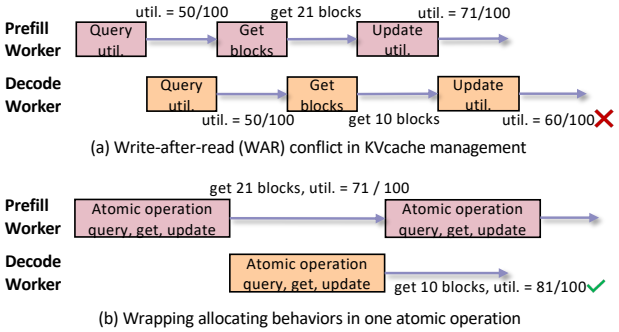

Figure 7: The atomic allocating operation avoids the WAR conflict.

Resource adjustment: solution. To solve the first overhead, we introduce a resident process to consistently hold the weights and KV cache during serving, avoiding the repeated loading of weights and copying of KV cache, as shown in Fig. 6(b). Then we share the pointer of the allocated space to the *prefill* process and *decode* process via the inter-process communication (IPC) [38] so that the *prefill* and *decode* workers can have access to the weights and KV cache through the pointer. In that way, the memory space of weights and KV cache is not released with the stopped worker, and the new workers can access the storage through IPC at negligible time cost. To hide the latency of IPC and initialization, *semi-PD* conducts the delayed switching, running under the new (x, y) only when the preparation step finishes.

To solve the second overhead, we propose asynchronous switching. We can renew two MPS processes directly and only kill the worker who has finished its iteration. Such an asynchronous behavior ensures there are always *prefill* and *decode* processes running in the system. Situations can arise temporally with an old *prefill* worker and a new *decode* worker running together, and vice versa, which are allowed in the system. This is attributed to the fact that MPS allows the resource percentage of all running processes to be larger than 100 percent, where each process will compete for the resources for a short while.

4.4 Unified Memory Manager

Two types of memory access are involved in *semi-PD*, *i.e.*, reading model weights and accessing KV cache. The unified memory manager can straightforwardly support the model weights since they are read-only. Thus, we mainly focus on the KV cache accesses. We follow vLLM [11] to use a paged storage for the KV cache, and the KV cache is accessed through the block table index. Once the block table index is determined, the access of the KV cache can be conducted without conflicts. However, conflicts happen when the *prefill* worker and the *decode* worker asynchronously allocate the KV cache. The *prefill* worker allocates the KV cache when a layer-wise computation is finished, and the *decode* worker allocates the KV cache for every layer in the PagedAttention [11] computation. The asynchronous manner of the two workers potentially brings write-after-read (WAR) conflicts to memory management. As illustrated in Figure 7, the allocating contains three steps: querying the memory utilization to see if there are free blocks, getting the blocks for KV cache storage, and updating the memory utilization. The WAR conflict happens when one worker immediately updates the utilization after it is queried. The result is that the querying worker updates the wrong memory utilization ratio based on the queried value. To address the challenge, we introduce the atomic operation for KV cache block allocating, *i.e.*, the memory utilization is locked until the update step finishes. Such an atomic allocating pattern ensures that memory utilization is correct against the two asynchronous processes.

4.5 Multi-GPU Scheme

Scheme. *semi-PD* supports serving LLMs across multiple GPUs, and the scheme is illustrated in Figure 8. Both *prefill* and *decode* workers reside on a single GPU, carrying their computation asynchronously. Thus, the inter-GPU communication happens asynchronously for the two workers. Specifically, within a TP group (*e.g.*, GPU 0 and GPU 1 in Figure 8), all the *prefill* workers communicate with each other to conduct the all-reduce operation. And within a PP group (*e.g.*, GPU 0 and GPU 2 in Figure 8), the *prefill* worker sends the activation to the *prefill* worker on the next GPU. The *decode* workers behave the same. Given a parallelism setting, the amount of data transferred remains the same as in a unified system. The limitation of such a scheme is the parallel patterns have to be the same for *prefill* and *decode* phases. However, we discuss in the following that the limitation is not an influential factor for serving performance.

Discussion. In a disaggregated system, the parallelism decoupling denotes that the *prefill* and *decode* workers can employ different parallel strategies (*e.g.*, TP=4 and PP=1 for the *prefill* worker while TP=2 and PP=2 for the *decode*

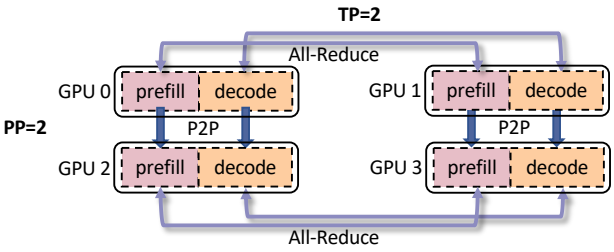

Figure 8: A TP=2 & PP=2 example with multi-GPU scheme.

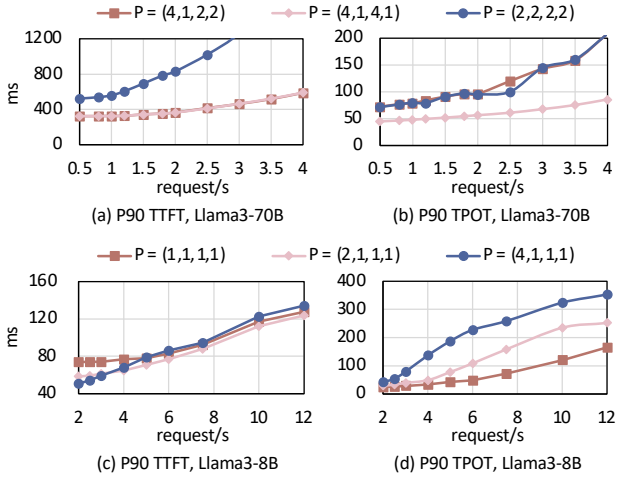

Figure 9: Performance of DistServe under different parallelism settings. The reported request rate is per GPU.

worker). We identify that the ratio of the number of GPUs of *prefill* workers and *decode* workers plays a more important role than different parallelism patterns.

Taking the commonly used TP and PP in inference as examples, their impacts on requests during the *prefill* and *decode* phases are similar. TP partitions a single request across multiple devices for parallel computation, which helps reduce request-level latency but introduces higher communication overhead. In contrast, PP enables parallel computation among different layers of distinct requests, which has lower communication overhead, thereby enabling the throughput to scale up with GPUs. Leveraging the insights from DistServe [17], TP is more beneficial for latency reduction under low request rates, whereas PP is required to enhance throughput under high request rates, which is a conclusion applicable to both *prefill* and *decode* phases.

As a supplement, we conduct experiments to validate the conclusion. We use $P = (t^{\mathrm{p}}, t^{\mathrm{d}}, p^{\mathrm{p}}, p^{\mathrm{d}})$ to represent the TP/PP degrees for *prefill* ($t^{\mathrm{p}}/p^{\mathrm{p}}$) and *decode* ($t^{\mathrm{d}}/p^{\mathrm{d}}$) phases, respectively. As shown in Figure 9, we test the performance of DistServe with Llama3-8B and Llama3-70B on the ShareGPT dataset. In Figure 9(a) and (b), for the 70B model, we observe that using $P = (4, 1, 4, 1)$ continuously outperforms other settings, indicating tensor parallelism is better than pipeline parallelism for both phases. It demonstrates that when we fix the GPU number of *prefill* and *decode* instances, decoupled parallelism cannot obtain performance benefits. Furthermore, Figure 9(c) and (d) depict that different ratios of GPU number of *prefill* workers and *decode* instances have a significant impact on performance.

5 Dynamic Adjusting Method

For latency-sensitive serving tasks, the goal of the serving system is to serve more requests adhering to the given service-level objectives (SLOs). For LLM serving, the SLOs refer to the latency constraints on TTFT and TPOT. Therefore, the resource partitioning between the *prefill* and *decode* workers (*i.e.*, (x, y)) should consider SLOs. Besides, the *prefill* and *decode* workloads change over time. A fixed resource partitioning (x, y) often leads to SLO violation, which arises from the mismatch between the dispatched resources and the dynamic workload demands. To address the issue, *semi-PD* enables an SLO-aware method to dynamically adjust the resource partitioning (x, y) following the workload changes. We first introduce a resident worker and a delayed switch mechanism to reduce the adjustment

Algorithm 1 SLO-aware adjusting algorithm

Input: iter #iteration index, (x_0, y_0) #current partition, $(S^{\text{p}}, S^{\text{d}})$ #user-defined SLOs, p^{SLO} #satisfied percentage defined in SLOs, *window_size*, *max_step*, *step_size* #algorithm configurations

1: **if** iter % *window_size* > 0 **then**
2: return (x_0, y_0)
3: **end if**
4: $(x, y) \leftarrow (x_0, y_0)$
5: TTFT, TPOT $\leftarrow$ get_observed_latency(p^{SLO})
6: update_estimate_model(x, y, TTFT, TPOT)
7: step $\leftarrow 0$
8: **if** TTFT SLO fails **then**
9: **while** step < *max_step* and estimate_ttft(x') $> S^{\text{p}}$ **do**
10: $(x, y) \leftarrow$ increase_x_ratio(x, y, *step_size*)
11: $x' \leftarrow x/(x+y)$
12: **end while**
13: **else if** TPOT SLO fails **then**
14: **while** step < *max_step* and estimate_tpot(y') $> S^{\text{d}}$ **do**
15: $(x, y) \leftarrow$ increase_y_ratio(x, y, *step_size*)
16: $y' \leftarrow y/(x+y)$
17: **end while**
18: **else**
19: return (x_0, y_0)
20: **end if**
21: return (x, y)

overhead. Based on that, we further design an SLO-aware algorithm to adjust the resource partitioning, which takes the given SLOs as inputs.

Algorithm. The proposed SLO-aware algorithm employs a feedback mechanism to adjust computational resources dynamically, implicitly accounting for the impacts of both input variation and model selection. The details are illustrated in Algorithm 1 and the inputs are outlined at the beginning. We conduct the iteration-level adjustment periodically based on a $window\_size$, and we collect the observed TTFT and TPOT within the window to update models for TTFT/TPOT estimation (lines 1-6). The algorithm directly returns the current partition when both TTFT and TPOT fail the SLOs, as there is no space for adjustment (line 19). Otherwise, we increase the SM ratio step by step for the phase with the failed latency and assess if the updated ratio satisfies the SLO (lines 9-10, 14-15). Given that $x, y \leq 100$, the increase in one ratio can be replaced by the reduction in the other if the increase leads to overflow. After that, we use the normalized SM ratio x' or y' to estimate TTFT or TPOT, considering that the two phases share 100% of the computational resource in total (lines 11, 16). The configuration *window_size* controls the adjustment frequency, and *max_step* and *step_size* decide the learning rate.

Latency modeling. At lines 9 and 15 in Algorithm 1, we need to model the relationship between TTFT and TPOT against the accessible SM ratio x and y, respectively. Since the two workers asynchronously handle requests, we model TTFT and TPOT separately. The TTFT of a single request consists of two parts of latency, *i.e.*, waiting latency, and *prefill* latency. On the other hand, we assume the requests are immediately batched for running upon arrival at the *decode* worker, hence excluding the waiting latency from TPOT. The more SMs a process can access, the more computational resources it can utilize in parallel, leading to a proportionally growing processing rate. Thus, we simply model the processing latency as

$$l_x = \frac{100}{x} \times l_{100}. \tag{1}$$

Here, l_x represents the processing latency under the SM ratio x for both *prefill* and *decode* phases. For TTFT, we apply an M/M/1 [39] queuing model to formulate the sum of *waiting* and *prefill* latencies as follows

$$w = \frac{1}{\mu_x - r}, \tag{2}$$

where r and μ_x denote the given input request rate and the *prefill* processing rate under SM ratio x, respectively. w is the sum of the waiting latency and the processing latency. Based on Eq. 1, we have $\mu_x = 1/l_x \propto x$, Eq. 2 is further

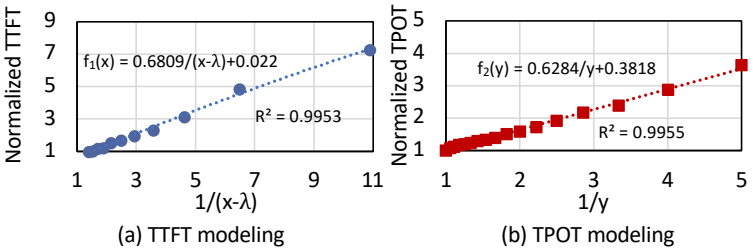

Figure 10: The inverse linear relationship of TTFT/TPOT against the SM accessible ratio. R^2 denotes the coefficient of determination, being closer to 1 is better. Data is collected with Llama3-8B and ShareGPT [40].

written as

$$w \propto \frac{1}{x-\lambda},\ \lambda = 100rl_{100}. \tag{3}$$

Therefore, the relationships between TTFT and x, TPOT and y are expressed as

$$TTFT_x = a_1\frac{1}{x-\lambda} + b_1,\ TPOT_y = a_2\frac{1}{y} + b_2. \tag{4}$$

The parameters $a_1, a_2, b_1, \lambda, b_2$ are learnable and computed based on the real-time observation of TTFT and TPOT. Note that we introduce the constant terms b_1, b_2 to model the constant overhead in the latency. Given (x, y), we utilize Eq. 4 to estimate TTFT and TPOT, so that we can quantify the impact of adjusting (x, y) during serving. Figure 10 validates the proposed latency modeling, where (a) shows $TTFT_x$ satisfying the linear relationship with $1/(x-\lambda)$, and (b) shows $TPOT_y$ satisfying the linear relationship with $1/y$.

6 Implementation

We build *semi-PD* on both DistServe [17] and SGLang [26] codebases to support instance-scale deployment, incorporating the system modifications described in Section 4 to enable disaggregated computation and unified storage. For the cluster environment, we modify the NVIDIA Dynamo [41] frontend and use vLLM [11] as the backend.

Implementation based on DistServe. We modify the grouped-query attention (GQA) with the optimized kernels from FlashAttention [42] for both phases. Besides, the first two general matrix multiplications (GEMMs) in the gated FFN are fused into one single GEMM. To support the Llama3.1 series [43] model, we also modify the RoPE kernel. Based on those modifications, *semi-PD* achieves comparable performance with state-of-the-art inference engines at the operator level. Note that we use all those optimized GPU kernels for DistServe as the baseline in the evaluation section.

To support uneven pipeline parallelism when the number of layers is not divisible by the parallelism degree (*e.g.,* Llama3.1-405B with 126 layers), we use the even pipeline parallelism so that the model can be deployed across 4 nodes with PP=4. Specifically, we dispatch the last two layers to the nodes in a round-robin manner.

semi-PD simply utilizes the first-come-first-serve (FCFS) scheduling for request processing, and the chunked-prefill mechanism is not supported for the *prefill* phase with the DistServe implementation. For the *decode* phase, we set a maximum batch size to control the batched request number.

Implementation based on SGLang. For the SGLang implementation, we develop a disaggregated frontend where *prefill* and *decode* instances operate as separate single-program-multiple-data (SPMD) processes that maintain consistent model parallelism strategies. We implement a state-copy mechanism between the *prefill* and *decode* message queues, enabling simultaneous request reception for the two phases, thereby eliminating inter-process communication overhead.

The system also utilizes first-come-first-serve (FCFS) scheduling for request processing. For the *prefill* phase, *semi-PD* employs the chunked-prefill mechanism to control the batched token number, avoiding overwhelming memory consumption brought by activation or prolonged batch processing latency. For the *decode* phase, we implement batch size controlling and incorporate the CPU-GPU overlap scheduling following SGLang.

Scalable Deployment. In single-instance deployment, *semi-PD* can be used for complete deployment of a single model, particularly suited for private or local deployment environments. The disaggregated computation in *semi-PD* effectively eliminates latency interference while the unified storage substantially improves GPU utilization efficiency, thereby enabling reductions in deployment costs.

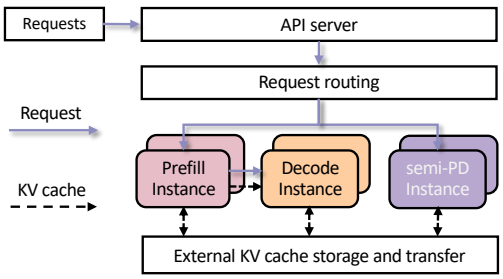

Figure 11: *semi-PD* instance applied in cluster-scale serving.

In cluster environments, *semi-PD* is treated as an instance selection together with *prefill* and *decode* instances, as illustrated in Figure 11. The requests are routed to the *prefill* instances or the *semi-PD* instances for the *prefill* phase processing. For the former case, requests are transferred from the *prefill* instance to the *decode* instance for computation, while for the latter, requests complete the *decode* phase computation within the same *semi-PD* instance. The request router dynamically routes requests between the disaggregated and *semi-PD* instances. Specifically, *semi-PD* is integrated into NVIDIA Dynamo, and on top of that, a router that collaboratively considers KV cache hit rate, GPU utilization, and waiting queue depth is designed to determine where the arrived requests go.

7 Evaluation

The evaluation of *semi-PD* comprises two environments: 1) instance-scale LLM serving and 2) cluster-scale LLM serving. For both environments, we evaluate *semi-PD* on various models and datasets, comparing the P90/P99 TTFT and TPOT to the baselines. We observe that as the input request rate grows, *semi-PD* maintains a relatively low latency for both TTFT and TPOT. Notably, *semi-PD* reduces the average end-to-end latency per request by 1.27-2.58× when serving DeepSeek series models. Also, considering the SLO attainment, *semi-PD* serves 1.55-1.72× more requests than the SOTA systems adhering to the SLOs on Llama series models. We also conduct experiments to evaluate the proposed dynamic adjustment algorithm and the low-overhead switching mechanism. Note that all request rates are per GPU in this section.

7.1 Setup

7.1.1 Testbed

The experiments involve four platforms. *Server A*: one node with eight NVIDIA A100 80GB SXM4 GPUs connected with pairwise NVLink [20]. *Server B*: four nodes with each equipped by eight NVIDIA A100 40GB SXM4 GPUs. The cross-node bandwidth is 200 Gbps. *Server C*: one node with eight NVIDIA H200 141GB SXM5 GPUs connected with pairwise NVLink [20]. *Server D*: one node with eight NVIDIA A800 80GB SXM4 GPUs connected with pairwise NVLink [20]. The software environment includes CUDA 12.1 [44], NCCL 2.18 [45], PyTorch 2.3.0 [46], etc.

7.1.2 Model

As listed in Table 1, for the implementation based on DistServe, we evaluate *semi-PD* using the Llama3/3.1 [47, 43] series models, with the model sizes of 8B, 70B and 405B. We deploy the 8B and 70B models on *Server A* while the 405B model on *Server B*. All parameters used for experiments are in FP16 precision. As demonstrated in Figure 9, considering both phases, TP=1 and TP=4 are respectively the best choices for the 8B and 70B models under latency constraints. Thus, we adopt TP=1 and TP=4 in the 8B and 70B evaluations. For the 405B model, we apply TP=8 and PP=4, with PP across different nodes.

For the implementation based on SGLang, we evaluate *semi-PD* using the DeepSeek [48, 49] series models. The DeepSeek-V2-Lite model has a parameter size of 16B, and is directly deployed on a single A100/A800 80GB GPU. The DeepSeek-V3 model has a parameter size of 671B and is deployed on *Server C* with TP=8 and FP8 precision.

7.1.3 Dataset

We choose representative workloads including chatbot, long context and math. The benchmarked datasets are ShareGPT [40], LongBench [50], and MATH-500 [51], respectively. For each workload, we sample from suit-

Table 1: Platform, benchmark, and model settings.

Enviroment	Platform	Codebase	Model	Parallelism	Dataset
Instance	1×A100 80GB SXM4 GPU (*Server A*)	DistServe	Llama3-8B	-	ShareGPT, LongBench
	4×A100 80GB SXM4 GPUs (*Server A*)		Llama3-70B	TP=4	
	32×A100 40GB SXM4 GPUs (*Server B*)		Llama3.1-405B	TP=8, PP=4	
	1×A100 80GB SXM4 GPU (*Server A*)	SGLang	DeepSeek-V2-Lite	-	ShareGPT
	8×H200 141GB SXM5 GPUs (*Server C*)		DeepSeek-V3	TP=8	MATH-500
Cluster	8×A800 80GB SXM4 GPUs (*Server D*)	Dynamo	DeepSeek-V2-Lite	-	Synthetic

Table 2: SLO settings in evaluation (TTFT/TPOT).

Model	ShareGPT		LongBench	
	Tight	Loose	Tight	Loose
Llama3-8B	0.3s/0.15s	0.4s/0.2s	2.25s/0.13s	3.0s/0.18s
Llama3-70B	0.85s/0.3s	1.1s/0.4s	6.0s/0.3s	8.0s/0.4s

able datasets and generate request arrival times using the Poisson distribution to emulate the real-world workloads. The datasets used for each model are illustrated in Table 1.

Regarding cluster-scale serving, we evaluate *semi-PD* on heterogeneous workloads to emulate the real-world scenario. For that purpose, we create a synthetic dataset consisting of 95% requests sampled from ShareGPT to represent the regular workloads, and 5% irregular requests owning long input length (∼4k compared to ShareGPT's 251 on average).

7.1.4 Baseline

For the Llama series models, we compare *semi-PD* against DistServe and vLLM, and for the instance-scale serving of DeepSeek series models, we compare *semi-PD* against SGLang. Furthermore, NVIDIA Dynamo is taken as the baseline for cluster-scale evaluation. The baselines are detailed as follows.

- **DistServe** [17]: One of the few disaggregated systems that release open-source codes. *semi-PD* uses the same GPU kernels and the same scheduling settings as DistServe. Specifically, *max_batch_size* is set to 512. When evaluated in instance-scale serving, DistServe is set to use one *prefill* instance and one *decode* instance (*i.e.*, 1P1D). Therefore, the occupied GPU number is doubled for DistServe in Table 1.
- **vLLM (v0.6.0)** [11]: A unified system accompanied by multiple open-source optimizations, we compare with its default implementation (**vLLM-D**) and SplitFuse implementation (**vLLM-S**). We set *max_batch_size*=512 for both vLLM-D and vLLM-S, and use *chunk_size*=1024 for vLLM-S.
- **SGLang (v0.4.4)** [26]: Another unified system with high-throughput optimization, providing support for DeepSeek-V3 serving upon the model's release.
- **Dynamo (v0.1.1)** [41]: A cluster-scale distributed inference serving system, featuring *prefill* and *decode* disaggregation, currently only supporting vLLM [11] as backend. When serving DeepSeek-V2-Lite in evaluation, Dynamo uses two *prefill* instances and six *decode* instances (*i.e.*, 2P6D), while we replace one of the *prefill* instances and three of the *decode* instances with four *semi-PD* instances (denoted as 1P3D4S).

7.1.5 SLOs

The SLOs are the constraints on TTFT and TPOT. The SLO settings in our experiments are in Table 1. We use the approximates of 7.5× and 10× of the single request latency as the tight and loose SLOs, respectively.

7.2 Instance-Scale Performance

To evaluate the overall performance of instance-scale serving, we mainly compare the P90 TTFT and TPOT across all the systems. We also present the results of P99 TTFT and TPOT to further demonstrate the efficiency of the design.

7.2.1 P90/P99 TTFT and TPOT Comparison

Llama series. As shown in Figure 12, other systems exhibit a sharp increase in TTFT or TPOT under high request rates, while *semi-PD* maintains stability across both latency metrics. For example, in Figure 12(a), vLLM-S and vLLM-D

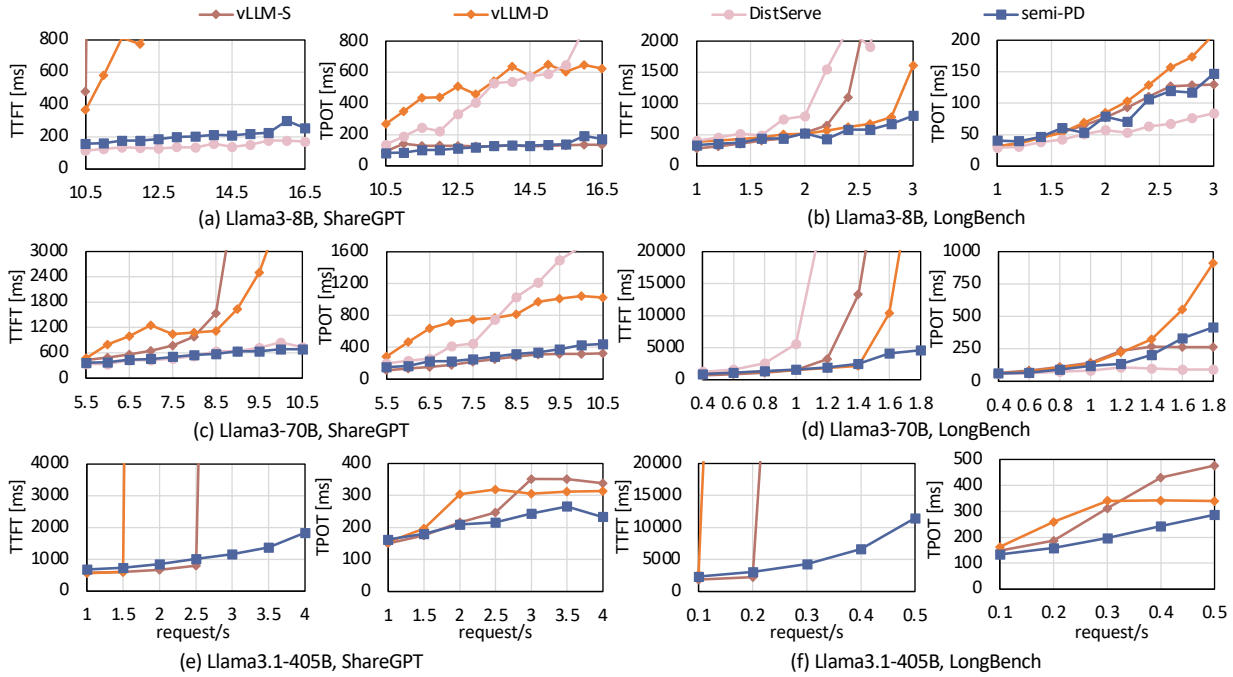

Figure 12: The P90 TTFT and TPOT comparison on Llama series models (lower is better). For Llama3.1-405B, we only compare *semi-PD* with vLLM-S and vLLM-D, as DistServe can not be deployed due to the storage limitation.

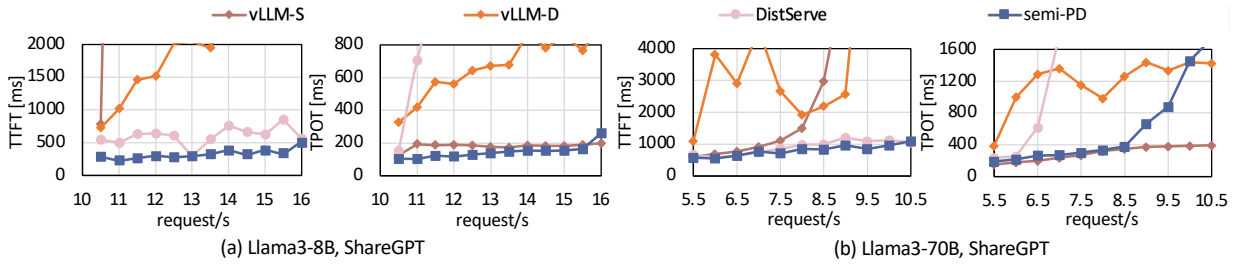

Figure 13: The P99 TTFT and TPOT comparison on Llama series models (lower is better).

pose a surge in the P90 TTFT, whereas DistServe increases drastically in the P90 TPOT. Meanwhile, *semi-PD* remains relatively low TTFT and TPOT when the input request rate exceeds 16 request/s. The surge in TTFT for vLLM-S is because of the prioritization of *decode* requests over *prefill* requests, and the latency interference becomes significant under higher request rates. Such latency interference is reflected in all the results. The surge in TPOT for DistServe is due to storage inefficiency mentioned in Section 3.

Similar trends are observed on the Longbench dataset. In Figure 12(b) and (d), the TTFT surge mainly arises from two aspects: 1) the *decode* worker of DistServe runs out of memory due to the storage imbalance, and the requests accumulate on the *prefill* worker. 2) At the beginning of serving, the *prefill* worker demands much more computational resources, but the disaggregated design fails to make adjustments at negligible cost. *semi-PD* addresses the two issues and avoids the TTFT explosion.

We compare the P99 latency results serving Llama-8B and Llama3-70B on ShareGPT. Figure 13 shows the P99 TTFT and TPOT, demonstrating a performance trend similar to Figure 12. The P99 metrics tend to exhibit more fluctuations across all systems. In fact, *semi-PD* delivers better performance in P99 latency, avoiding latency explosion due to latency interference and storage imbalance. *semi-PD* is the only one that maintains smooth TTFT and TPOT, which can handle over 11 request/s for Llama3-8B and 8 request/s for Llama3-70B. Without loss of generality, we choose to use the P90 latency as the major metric.

DeepSeek series. The results are depicted in Figure 14. When serving DeepSeek-V2-Lite on ShareGPT, *semi-PD* delivers lower P90/P99 TTFT and TPOT under high request rates, demonstrating the ability to serve more requests

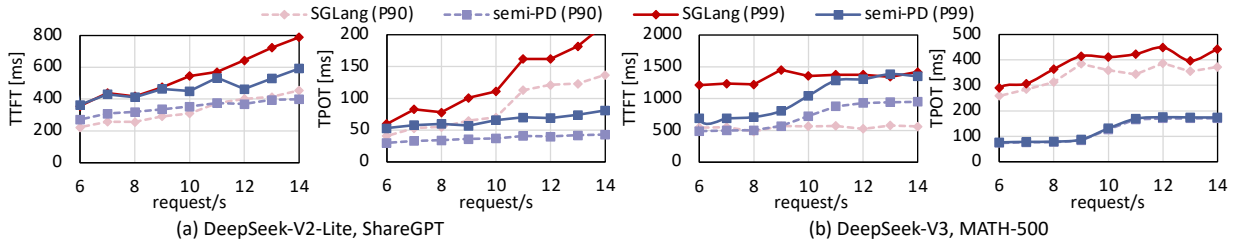

Figure 14: The P90/P99 TTFT and TPOT comparison on DeepSeek series models (lower is better).

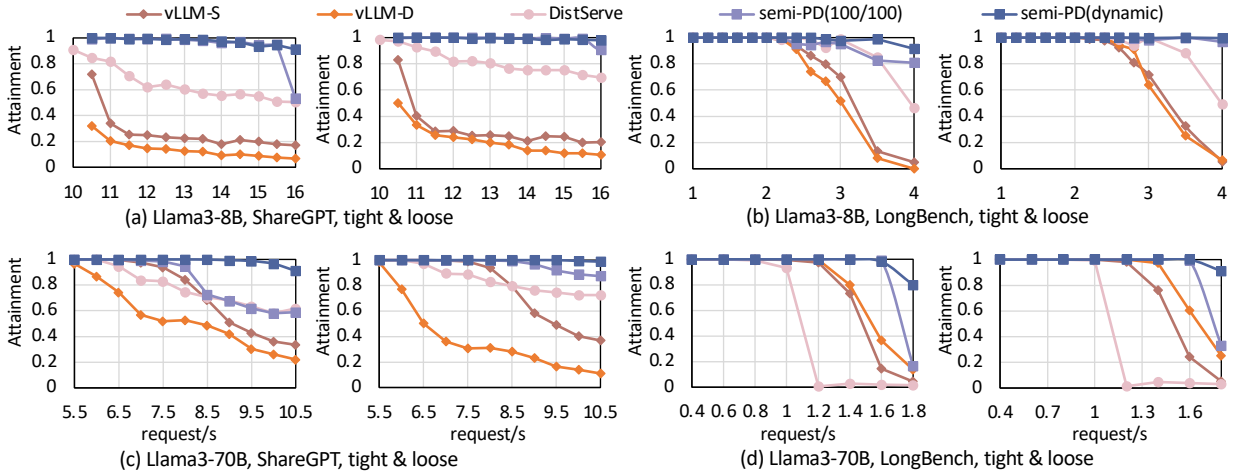

Figure 15: The SLO attainment comparison (higher is better). For (a)-(d), the tight/loose SLO results are on the left/right.

adhering to latency constraints. Although *semi-PD*'s P90 TTFT is slightly higher when serving DeepSeek-V3 on MATH-500, its P99 TTFT remains at a low level under high request rates. Notably, the TPOT of *semi-PD* is far better than SGLang's. Furthermore, we observe that *semi-PD* brings 1.27-2.40× and 1.49-2.58× speedups in the average end-to-end latency per request when serving DeepSeek-V2-Lite and DeepSeek-V3, respectively.

7.2.2 SLO Attainment

Given SLOs, the proposed dynamic adjusting method enables the real-time resource adjustment between *prefill* and *decode* phases, thereby enhancing the SLO attainment. We show the attainment improvement brought by the dynamic adjusting method for evaluation. We use two different SLOs, tight and loose, as demonstrated in Section 7.1.5, to capture a more comprehensive attainment comparison result. We take *semi-PD* with $(x, y) = (100, 100)$ as the baseline dynamic method, which hands in the responsibility of resource partition to the CUDA scheduler, and performs uncontrollable dynamic adjustments between two workers. We denote the implementation with the proposed dynamic adjusting method as *semi-PD* (dynamic).

Figure 15 presents the SLO attainment of all the systems. Both *semi-PD* (100, 100) and *semi-PD* (dynamic) outperform other baselines, maintaining the attainment at a high level as the request rate increases. *semi-PD* (dynamic) further improves the SLO attainment compared to *semi-PD* (100, 100), thanks to its SLO-aware algorithm as the adjustment guidance. On average, *semi-PD* (dynamic) achieves 1.55×, 1.72×, 1.62×, 1.11× higher request rate over vLLM-S, vLLM-D, DistServe, *semi-PD* (100, 100), respectively, while maintaining the SLO attainment larger than 90%.

7.3 Cluster-Scale Performance

The results in Figure 16 demonstrate that the integration of *semi-PD* instances reduces both TTFT and TPOT against heterogeneous workloads. Notably, with *semi-PD*, the system avoids the TTFT surge under low request rates, and achieves more than 2× of the TPOT reduction. For the baseline Dynamo implementation, given eight GPUs, 2P6D is the optimal solution under the regular requests sampled from ShareGPT. However, such a static resource partition fails to keep up with sudden workload changes brought by those irregular requests. To adapt to the changes, a

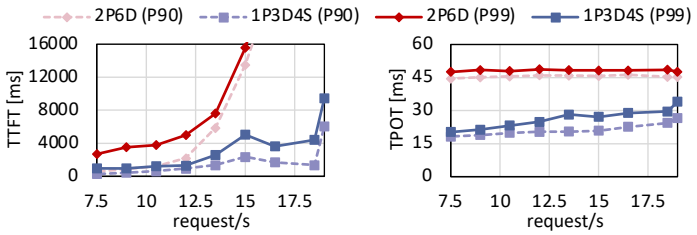

Figure 16: The cluster-scale serving latency comparison (lower is better). 2P6D denotes using pure Dynamo with two *prefill* instances and six *decode* instances, while 1P3D4S denotes using one *prefil* instances and three *decode* instances with Dynamo and four extra *semi-PD* instances.

disaggregated system requires relatively high overhead for adjustment, as mentioned in Section 3. On the other hand, the *semi-PD* instances enable low-overhead resource adaptation to workload changes, while maintaining interference-free computation between the two phases. When integrated into disaggregated systems, *semi-PD* instance effectively functions as an elastic buffer for resource adjustment. Therefore, by replacing a portion of the instances with *semi-PD* instances, the system can handle the heterogeneous workloads better.

8 Conclusion

In this paper, we introduce *semi-PD*, an efficient LLM serving system characterized by disaggregated computation and unified storage. *semi-PD* follows the disaggregated system to avoid latency interference between *prefill* and *decode* phases, and further addresses its drawbacks of inefficient storage. Moreover, by developing a low-overhead worker switch mechanism, *semi-PD* can adjust the resource partition between the two phases at negligible cost. Based on that, we propose an SLO-aware algorithm for *semi-PD*, optimizing the SLO attainment by adjusting the resource partition during serving. Integrated with the designs mentioned above, *semi-PD* achieves much lower latency than the SOTA systems, reducing the average end-to-end latency per request by 1.27-2.58× on DeepSeek series models, and serves an average of 1.55-1.72× more requests under the given SLOs on Llama series models.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[2] Daniel Adiwardana, Minh-Thang Luong, David R So, Jamie Hall, Noah Fiedel, Romal Thoppilan, Zi Yang, Abhijit Kulshreshtha, Gaurav Nemade, Yifeng Lu, et al. Towards a human-like open-domain chatbot. *arXiv preprint arXiv:2001.09977*, 2020.

[3] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott, Kurt Shuster, Eric M Smith, et al. Recipes for building an open-domain chatbot. *arXiv preprint arXiv:2004.13637*, 2021.

[4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.

[5] Alex Svyatkovskiy, Shuo Deng, Shengyu Fu, and Neel Sundaresan. Intellicode compose: Code generation using transformer. *arXiv preprint arXiv:2005.08025*, 2020.

[6] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

[7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carson Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Jakub Lang, Kenny Christopher, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[8] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[9] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiao-Ping Zhang, Yuhan Dong, and Yu Wang. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.

[10] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022.

[11] Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[12] NVIDIA. Fastertransformer: About transformer related optimization, including bert, gpt. [Online], 2017. `https://github.com/NVIDIA/FasterTransformer`.

[13] Amey Agrawal et al. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[14] Connor Holmes et al. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671*, 2024.

[15] Cunchen Hu et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.

[16] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, and Saeed Maleki. Splitwise: Efficient generative llm inference using phase splitting. In *51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.

[17] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[18] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. Exegpt: Constraint-aware resource scheduling for llm inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 369–384, 2024.

[19] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving. 2024.

[20] NVIDIA. Nvlink and nvlink switch, 2024. `https://www.nvidia.com/en-us/data-center/nvlink/`.

[21] Hugo Touvron et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, 2017.

[23] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[24] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, , and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[25] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024.

[26] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, 2023.

[27] Neal Vaidya et al. Optimizing inference on large language models with nvidia tensorrt-llm, now publicly available. [Online], 2023. `https://github.com/NVIDIA/TensorRT-LLM`.

[28] InternLM. Lmdeploy is a toolkit for compressing, deploying, and serving llms. [Online], 2024. `https://github.com/InternLM/lmdeploy`.

[29] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, et al. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024.

[30] Bingyang Wu et al. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.

[31] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in serving large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[32] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient llm scheduling by learning to rank. *arXiv preprint arXiv:2408.15792*, 2024.

[33] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, et al. Nanoflow: Towards optimal large language model serving throughput. *arXiv preprint arXiv:2408.12757*, 2024.

[34] Tri Dao et al. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

[35] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on gpus. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[36] Reza Yazdani Aminabadi et al. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.

[37] NVIDIA. Nvidia mps, 2022. `https://docs.nvidia.com/deploy/mps/index.html`.

[38] NVIDIA. Inter-process communication. `https://docs.nvidia.com/drive/drive_os_5.1.6.1L/nvvib_docs/index.html#page/DRIVE_OS_Linux_SDK_Development_Guide/Graphics/nvsci_nvsciipc.html`.

[39] John F Shortle, James M Thompson, Donald Gross, and Carl M Harris. *Fundamentals of queueing theory*, volume 399. John Wiley & Sons, 2018.

[40] Sharegpt teams. Sharegpt. [Online], 2023. `https://sharegpt.com`.

[41] NVIDIA. Nvidia dynamo: A datacenter scale distributed inference serving framework. [Online]. https://github.com/ai-dynamo/dynamo.

[42] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[43] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[44] Nvidia. Cuda toolkit. [Online], June 2024. `https://developer.nvidia.com/cuda-toolkit`.

[45] Ammar Ahmad Awan, Khaled Hamidouche, Akshay Venkatesh, and Dhabaleswar K Panda. Efficient large message broadcast using nccl and cuda-aware mpi for deep learning. In *Proceedings of the 23rd European MPI Users' Group Meeting*, pages 15–22, 2016.

[46] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[47] Meta. Introducing meta llama 3: The most capable openly available llm to date, April 2024.

[48] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[49] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437v1*, 2024.

[50] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

[51] HuggingFaceH4. Math-500. [Online]. https://huggingface.co/datasets/HuggingFaceH4/MATH-500.